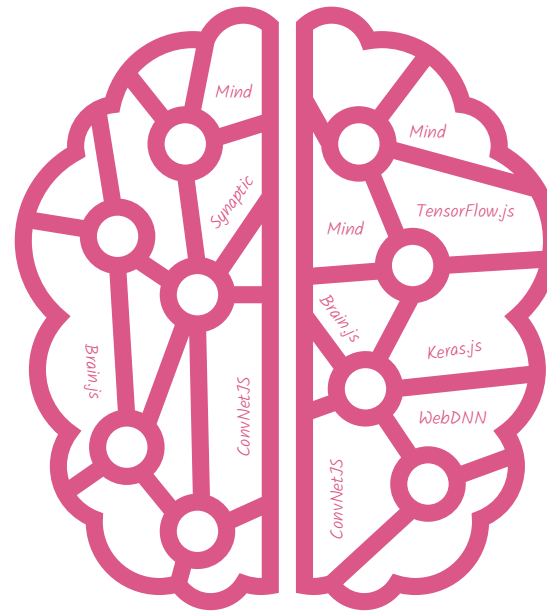
A Few Facts About Deep Learning in a Web Browser



Data privacy has given an impetus to browser based deep learning, as it helps data remain in the vicinity of a device. Another factor driving its demand is quicker page loading.

Deep learning in a browser may seem as odd as a camel being 'the ship of the hills'. However, the advent of platforms such as WebGL (JavaScript to render interactive 2D) and WebGPU (graphics and application acceleration) has made it possible to run deep learning (DL) programs on a Web browser — mostly routed through gaming software and platforms. Although it is possible to have both learning (from data) and inference (e.g., whether it will be 'partly sunny' tomorrow), the latter technically happens in the Web browser based on the model deployed remotely on the device. One major driving force for this is the increased awareness about data privacy, as it helps data remain within the vicinity of the device itself. Quicker response and page load time is another driving factor.

The concept of running deep learning tasks on Web browsers originated around 2015. Andrej Karpathy developed ConvNetJS during his stint as a PhD student at Stanford, which was the very first incarnation of DL in a Web browser. Some of its notable successors are WebDNN, Mind and Keras.js. Around three years back, in early 2018, Google propelled the movement by developing TensorFlow.js (tfjs), which can be termed the defining moment of this journey so far.

Features

Before delving into more details, let's look at some of the salient features of browser based DL, which continues to evolve at a very quick pace. In fact, it is safe to say that the era of browser based DL has just begun, and that interactive gaming is the main force driving its evolution.

Of the various steps in a DL workflow, decision making, which is based on trained data sets (what we call inference) dominates the scene. Although training is still feasible, people prefer to train globally and infer locally in the Web browser.

To infer something, the first step is to load the trained model on to the browser. This is called the model warm up step, and is perhaps the most sluggish in the entire inference process.

It has been experimentally determined that TensorFlow.js is x1 to x2 slower (with respect to the CPU) compared to its native counterpart, when run on a browser. Interestingly, it has also been found that tfjs running in an integrated graphics card equals or even outperforms the native TensorFlow on the CPU for the same inference task [Reference: *https://arxiv.org/abs/1901.09388*]. Also, the inference task does not clog up the CPU completely. Experimental results show it tops up around 80 per cent of the CPU.

Deep learning in a browser involves three distinct steps.

Model import: This is the most time-consuming step of importing the already trained models into the browser. TensorFlow.js has the tfjs converter (*https://github.com/tensorflow/tfjs/tree/master/tfjs-converter*) to load the natively trained TensorFlow models into the browser and make it suitable for inference.

Warm up: This is the next step, where the browser consumes the trained model. Experimental results show TensorFlow.js is ahead in the race.

Inference: This is the decision-making step, where the inference is made locally. Basically, the warmed-up model is